Recovering Repeated Structure in Flat 2D Rectilinear Layouts: A Geometry-First Framework Unifying Array Detection and Hierarchy Recovery

July 2026

Contents

Draft — internal working version. All numbers are from measured runs of the implemented C++/Boost.Geometry system (synthetic benchmarks and a real `gdstk` *GDS round-trip). The one remaining open item — running the identical harness on a real SKY130/ORFS* `.gds` *crop — is marked* `[TBD]`*.*

Abstract

A flattened integrated-circuit mask, a tiled facade, a decorative pattern — these arrive as a single flat pool of polygons, but a person looking at them sees repeated structure immediately: *that is an array, this motif recurs, those rows are the same cell mirrored.* The generative structure was there upstream, as step-and-repeat and cell-reference metadata, and it is worth recovering — for compression, verification, and layout analysis — but flattening threw it away, leaving only geometry.

This paper is organized around one empirical observation and the algorithm it suggests. The observation: **repetition in a 2D layout shows up as repetition on 1D lines.** If a rectangular region is an array with horizontal step `dx`, then *every* horizontal scanline crossing it is periodic in `x` with period `dx` — so line-wise 1D periodicity is a *necessary condition* for a 2D array, cheap to test and cheap to falsify. The algorithm this suggests: **serialize each scanline into a string of tokens and let fast string-repetition algorithms find the repeats.** Periodic *runs* in the string are the fingerprint of a lattice-aligned array; *repeated substrings* (the same motif recurring with unrelated content between occurrences) are the fingerprint of a scattered, non-aligned cell — which is exactly the hierarchy-recovery case. String detection is the engine of the whole method.

The engine is only a filter, and we are explicit about that: a string match over serialized geometry is *lossy* — serialization order is a choice, tolerance binning can preserve a symbolic period while the physical lattice drifts — so **the string stage proposes and canonicalized geometric equality proves.** A minimum-description-length (MDL) objective then decides which certified repeats are

worth promoting to cells. The central claim is that **array detection is exactly the one-level, lattice-aligned special case of hierarchy recovery**, demonstrated (not merely argued): on the same flattened layout, the dedicated array detector and a one-level run of the hierarchy recoverer followed by a lattice fit produce **identical lattice parameters**.

The framework is implemented in C++17 on Boost.Geometry. It reproduces three previously-diagnosed bug classes as passing regression tests; recovers decorative and standard-cell arrays (including directly-abutted zero-gap geometry a single encoding provably cannot handle); recovers D4-oriented (mirrored/rotated) instances as one cell; and passes both a synthetic and a **real-GDS** hierarchy-erasure benchmark with exact flatten-equality. As a placement-list hierarchy it achieves 3.3–3.5× compression on the synthetic datapaths and 3.80× on the real-GDS round-trip; adding explicit top-level **array nodes** raises those to 9.3–19.2× on the synthetic datapaths and 20.86× on the real-GDS round-trip, while still recovering the leaf cell in every case. (An earlier, higher compression figure came from an over-covering verification bug; an exact rectangle-multiset check both fixed it and honestly lowered the placement-list number. Array nodes recover the valid dense-lattice compression without reintroducing overlap.) Performance is linear at roughly 3 μs per rectangle; 1.58 M rectangles recover in about 4.8 s single-round on one core. Throughout, where ground truth exists we report **agreement, not accuracy**: the recovered hierarchy is one of many valid decompositions of the same flat geometry.

---

1. Introduction

1.1 A layout that has forgotten its own structure

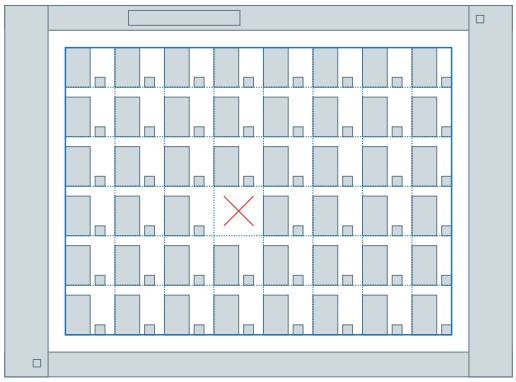
Consider Figure 1. On the left is a decorative grid: a small two-rectangle motif repeated 48 times on a regular lattice, wrapped in a non-repeating border, a title bar, and corner ornaments. A person sees the array in a fraction of a second. The file does not: it is a flat pool of ~100 axis-aligned rectangles with no record that any of them are copies of any others. On the right is the harder case — the same kind of motif, but now scattered at irregular positions, some copies clustered into a larger group, most of the plane filled with unrelated shapes. A person still sees "that little L-shape keeps showing up," but there is no grid to lock onto.

This is the situation after **flattening**. A modern IC layout is authored hierarchically — a designer defines a small cell (a logic gate, a bit-slice, an SRAM bit) and place-and-route instantiates it thousands or millions of times, often on regular rows and arrays recorded compactly as array references (AREFs). That hierarchy is what keeps a billion-transistor GDSII/OASIS file a manageable size. But many downstream operations — DRC region extraction, third-party IP analysis, reverse-engineering a flattened block, mask-data-prep — receive geometry that has *already* been flattened: the cell and array references are expanded into raw polygons and the metadata is gone. The same happens outside ICs: a decorative tiling, a rendered pattern, or a scanned facade arrives as flat geometry with its generative structure only implicit.

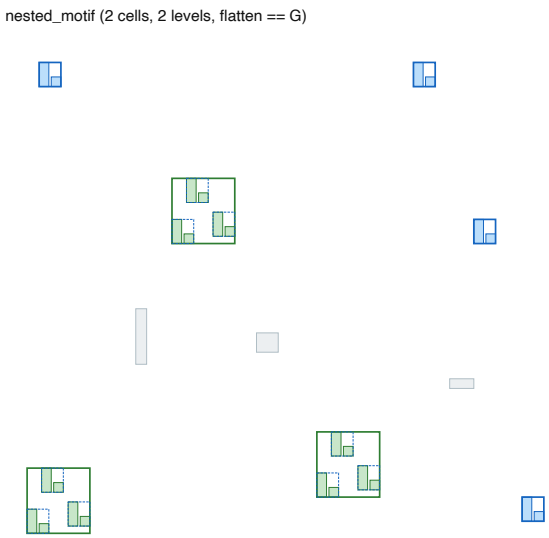
1.2 What we recover, and why it is worth recovering

We recover the lost structure directly from the geometry, across the full spectrum from Figure 1(a) to (b):

- For a genuine **array**, we report `(tile, step-x, step-y, count)` plus a short defect list.



(a) A lattice-*aligned* array: one motif on a regular 8×6 grid.



(b) The *unaligned* case: the same motif scattered off any grid, some copies grouped into a larger super-cell (green), the rest clutter (gray).

Figure 1: The two regimes this paper unifies. In (a) the repeated cell sits on a uniform lattice, so its repetition is *periodic*; in (b) the repeated cell recurs at arbitrary positions with unrelated content between occurrences, so its repetition is a *repeated substring* rather than a period. Both are flat rectangle pools with no metadata; both are shown here with the recovered structure overlaid. Array detection handles (a); hierarchy recovery handles (b) and contains (a) as its lattice-aligned special case.

That is dramatically smaller than the expanded polygon set, and — just as usefully — it makes the array's *defects* first-class: a missing instance, a stray polygon overlaid on an otherwise-perfect cell.

- For the general case we recover a compact **hierarchy**: a small library of cell definitions plus instance placements (and a residual for whatever does not repeat), such that flattening the hierarchy reproduces the original geometry exactly. This re-compresses a flattened layout and can expose design intent the flattening erased.

Both are valuable precisely because the repetition is usually *partial*: a regular region embedded in clutter, borders, routing, and localized defects, not a globally periodic image.

1.3 The idea, in one paragraph

The whole method follows from reducing a 2D problem to many 1D ones. Repetition in the plane leaves a shadow on every line that crosses it (§3): a 2D array forces 1D periodicity on each scanline through it, and a scattered repeated cell forces the *same substring* to appear at several places along a scanline. So we serialize each scanline into a string of tokens and run **string-repetition algorithms** — periodic-run detection for arrays, repeated-substring / grammar-induction for scattered cells — as the detection engine (§4). Because serialization is lossy, the string stage only *proposes* candidates; **canonicalized geometric equality certifies them** and an MDL objective decides which certified repeats become cells (§5). Array detection is then literally hierarchy

recovery restricted to one level whose instances happen to land on a lattice (§5.6, §7.2).

1.4 Contributions

- **A geometry-first framework** spanning rigid lattice arrays through arbitrary nested hierarchy, built on two shared mechanisms: canonicalized geometric verification as the *sole* certifier, and an MDL compression-gain objective as the promotion rule.
- **The 1D-necessary-condition framing made operational**: line-wise periodicity is a cheap necessary test for a 2D array, and its natural computational form is a string problem. We make the string layer the detection engine and are explicit that it is a filter, not a proof.
- **The array/hierarchy split cast as a string-tool distinction**: periodic *runs* fit adjacent-on-a-lattice array instances; *repeated substrings* fit cells scattered with unrelated content between occurrences (§4.2). This is the non-alignment gap between the two problems, stated in the vocabulary that solves it.
- **The repeat-length drop-off as a scale prior** (§3.3, §5.4): grow a motif until the number of surviving repeats drops off a cliff, and read the cell scale off the cliff — a *size* signal that complements the array's *period* signal.
- **Two complementary symbolic encodings** (occupancy-interval and edge-boundary) that together handle both gapped and directly-abutted (zero-gap) geometry — a case a single encoding provably fails on.
- **A period-phase reconciliation formulation** as clustering over physical period, modular phase, and spatial support, with a center-pruning step that defeats the transitive weak-link failure mode.
- **A defect model** (`missing = expected − observed`, `extra = observed − expected`) with exact geometry, correctly attributed to physical positions across defects — reproducing and guarding against three previously-diagnosed bug classes as regression tests.
- **D4 orientation support** (the 8 axis-preserving reflections/rotations), so mirrored standard-cell rows recover as one cell rather than eight spurious ones.
- **An exact O(n) rectangle-multiset verification used as a correctness gate**, not merely a metric: it caught an over-covering recovery bug that a weaker containment check had rubber-stamped, and forced the honest compression numbers down.
- **Array-node compression for recovered lattice placements**: after exact hierarchy recovery, top-level placements of the same cell/orientation are fit to conservative 2D or 1D lattice nodes, recovering dense-array compression as an explicit representation rather than as invalid greedy nesting.
- **An empirical demonstration of the unification**: array detection realized as one-level hierarchy recovery plus a lattice fit, the two paths producing identical lattice parameters (§7.2).

---

2. Problem Statement

2.1 Array detection

Input. A collection of axis-aligned rectangles in the plane. (The rectilinear scope means a rectangle is the only primitive we need; a fractured polygon is several rectangles.)

Target. A candidate array is a tuple **A = (B, u, v, T)** where `B` is a rectangular bounding box, `u = (dx, 0)` and `v = (0, dy)` are orthogonal lattice vectors, and `T` is the tile geometry of one fundamental cell. The **exact array condition** [v4, §1.2] is

$$G \cap B \;=\; \bigcup_{i,j} \left((T + i\,\mathbf{u} + j\,\mathbf{v}) \cap B \right) \qquad \text{(after coordinate snapping and canonicalization)}$$

The canonicalization qualifier is load-bearing: comparing raw, unsnapped geometry would reject valid arrays for representational reasons. The detector seeks a **primitive** tile (the smallest verified repeating unit) but may report a **supercell** when primitive recovery is ambiguous; a coincidental match that survives no candidate period is a **pseudo-array** and is discarded [v4, §2.4].

2.2 Hierarchy recovery

Input. Flat polygon geometry `G` (again, rectangles under our scope).

Output. A hierarchy $H = (C_1..C_m,\ I_1..I_k,\ R)$ of cell definitions, instance placements, and residual geometry, such that

$$\text{flatten}(H) = G \quad \text{within tolerance } \varepsilon$$

minimizing

$$\text{cost}(H) = \sum_i \text{cost}(\text{body}(C_i)) + \sum_j \text{cost}(\text{inst}(I_j)) + \text{cost}(\text{residual}(R))$$

Cells may reference smaller cells (nested hierarchy), which is how "arrays inside arrays" are represented [HR, §1.1]. Candidate generation is heuristic; only `flatten(H) = G`, checked by canonicalized equality, certifies a hierarchy.

The essential difference from §2.1 is **alignment**. An array's instances are adjacent on a uniform grid; a hierarchy's instances are anywhere — Figure 1(b). Array detection can lean on a single period and phase; hierarchy recovery cannot, and must find the same geometry recurring at unrelated offsets. §3 and §4 are about turning that difference into two different (but adjacent) string problems.

2.3 Why this is hard: partiality and representation slack

Two properties make this different from classic periodicity detection.

- **Partiality (Figure 2).** The layout is *not* globally periodic. An array occupies a rectangular sub-region while borders, routing, filler, and unrelated blocks surround it; multiple distinct arrays may coexist. A global-periodicity method (autocorrelation, a global Fourier peak) assumes the whole image repeats and so answers the wrong question. Repetition evidence must be gathered *locally* so that partiality is the normal case, not a failure mode.
- **Representation slack.** The same logical geometry can be drawn many ways — fractured versus merged, different winding, near-duplicate coordinates from floating-point noise. Any comparison naive about representation will reject valid repeats for non-geometric reasons. Every acceptance decision must therefore route through a **canonicalized** comparison.

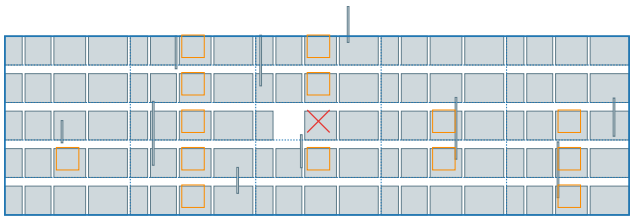


Figure 2: Partiality made concrete. A standard-cell array (blue region, recovered lattice dashed) is embedded in clutter: vertical routing wires cross the array, orange squares mark stray polygons overlaid on otherwise-clean cells (`extra`), and the red × marks one genuinely missing cell (`missing`). A global-periodicity method answers the wrong question here; the period must be found *locally*, line by line, and defended against the clutter.

The framework confronts both directly: evidence is gathered line-wise and locally (§3–§4) so partiality is natural, and every acceptance routes through canonicalized geometric comparison (§5.3) so representation slack is neutralized.

2.4 Non-uniqueness, stated up front

A flat layout generally admits **many** valid hierarchical decompositions. `ABABABAB` flattens identically whether the source was one cell `AB` × 4, alternating `A`/`B`, or `ABAB` × 2. The target is therefore *a compact, geometrically valid hierarchy*, not *the designer's original hierarchy*. Where evaluation compares against a known source, we report **agreement**, not **accuracy** [HR, §1.3]. This is not a caveat bolted on at the end; it shapes the evaluation (§7) and the limitations (§8).

2.5 Assumptions and scope

Shared by both problems [v4, §1.1; HR, §1.2]:

- **Axis-aligned, rectilinear (Manhattan)** geometry with an orthogonal two-vector lattice — not arbitrary oblique lattices.
- **Single-layer** geometry (multi-layer/multi-class is future work).
- **Fractured polygons** are expected; verification must tolerate one logical shape represented as several rectangles.
- **Exact repetition** (within tolerance ε) is the primary target; bounded-defect matching is a defined extension, not the default.
- Holes and overlapping polygons are out of scope for the initial version.
- **Transform scope.** Translation-only by default, deliberately **extended to the dihedral group D4** — the 8 axis-preserving orientations (rotations by 0/90/180/270°, each optionally mirrored) — because place-and-route flows routinely mirror alternating standard-cell rows to share a power rail. Under translation-only canonicalization a mirrored instance would register as unrelated geometry; instance records therefore carry an **orientation field**, not just a position [HR, §1.2].

---

3. The Key Idea: Repetition Lives on 1D Lines

Before any algorithm, the observation that makes the algorithm cheap. We build it up in the order it matters: the necessary condition for an array, why hierarchy needs a different primitive, and how to fix the cell *scale*.

3.1 A 2D array forces 1D periodicity on every line

Suppose a rectangular region `B` is a perfect array: tile `T` repeated at every lattice point $T + i\,\mathbf{u} + j\,\mathbf{v}$ with $\mathbf{u} = (dx, 0)$, $\mathbf{v} = (0, dy)$. Take any horizontal line `y = c` that crosses `B`. Because the layout is unchanged under translation by $\mathbf{u}$ (within `B`), the pattern of filled and empty intervals that the line sweeps is **exactly periodic in `x` with period `dx`**. The same holds vertically: every vertical line through `B` is periodic in `y` with period `dy`. This is immediate, but it is the whole lever:

> **If a region is a 2D array with step `(dx, dy)`, then every horizontal scanline through it is 1D-periodic with period `dx` and every vertical scanline is 1D-periodic with period `dy`.**

So "is there an array here, and what is its step?" reduces to "are many aligned 1D lines periodic, and with what period?" — a question about lines, not about the plane. Crucially the condition is **necessary, not sufficient**: many aligned lines being periodic is strong evidence of an array but does not *prove* one, because a 1D line integrates away the second dimension (two different 2D patterns can share the same line signature). That gap is deliberate and is where geometry re-enters (§3.4, §5.3). The payoff is asymmetric and worth naming: the necessary condition is *cheap to test and cheap to falsify* — a single non-periodic line, or an incompatible period, kills a candidate before any expensive 2D geometry is touched.

Partiality (§2.3) drops out for free: we never require the *whole* line to be periodic, only a *run* of it. A line crossing an array embedded in clutter is periodic over the array's `x`-extent and arbitrary elsewhere. Finding the periodic run within a mostly-non-periodic line is precisely a local, line-wise question.

3.2 Hierarchy needs a different primitive: repeated substrings, not periods

Now the unaligned case, Figure 1(b). The cell still recurs, but its instances are *not* adjacent and *not* on a grid. A scanline crossing several instances does not see a period — it sees the cell's signature, then unrelated content, then the cell's signature again, then more unrelated content: `… A … A … A …` with arbitrary gaps. That is not a periodic run; it is the **same substring appearing several times at unrelated offsets**.

This is the single conceptual difference between the two problems, and it is a difference in the *kind of repetition*:

	array	hierarchy
instances are	adjacent on a lattice	scattered anywhere
line signature is	a **periodic run** (`ABCABCABC`)	a **repeated substring** (`ABC…ABC…ABC`)
pinned by	one period + phase	occurrences of a motif + its length

Everything else — canonicalized certification, the MDL objective, defect handling — is shared.

The split is entirely about aligned-and-periodic versus scattered-and-repeated, which is why §4 can treat both with the *same* serialization and two adjacent families of string algorithm.

3.3 Fixing the cell scale: the repeat-length drop-off

The scattered case raises a question the array case never does: **how big is the cell?** On a lattice the period fixes the tile extent. Off a lattice there is no period to read a size from, and a repeated substring is ambiguous about its own boundary — `A`, `AB`, and `ABC` may all recur, and only one of them is the natural cell.

The signal that fixes the scale is a *drop-off in repetition as the motif grows.* Take a small recurring motif and grow it — add one more neighboring token, then another. While the growing motif is still *inside* the true cell, every occurrence grows in lockstep and the repeat survives: the count of surviving copies stays high. The moment the motif grows *past* the cell boundary — reaching into the per-instance clutter that surrounds each occurrence — the occurrences stop agreeing and the number of surviving repeats **falls off a cliff.** In words: grow the motif until the repeat suddenly stops; the size at the cliff is the cell scale.

Made quantitative over candidate motif length `L` [HR, §3]:

```
R(L) = number of distinct repeated motifs of length ≥ L
F(L) = total occurrence count summed over those motifs
C(L) = total MDL compression gain (§5.4) summed over those motifs
```

The expected shape is many short repeats, progressively fewer at longer lengths, and a sharp knee at the layout's real cell scale — as `L` climbs, `F` holds and then drops, while `R` fragments upward as the growing neighborhood dissolves into non-repeating surroundings. Both are the same cliff seen from two sides. This is a **scale prior**, not a decision procedure: it narrows *where to look* for the cell boundary; it does not by itself certify a cell. Which candidates actually become cells is decided by geometric verification and compression gain (§5.4). We implement the curve as a first-class, inspectable analysis and expose promotion-at-the-drop-off as a selectable criterion, with MDL gain the default (§5.4).

3.4 Why "necessary, not sufficient," and where geometry comes in

Both signals above are *filters.* Line periodicity is necessary for an array but does not prove one; a drop-off suggests a cell scale but does not prove a cell. The reason is the same in both cases: a 1D line, or a symbolic token stream, has thrown away information that only 2D geometry holds. Two consequences run through the rest of the paper. First, the cheapness of the filters is the point — they let us *propose* structure over a whole layout in linear time and discard most of the plane before touching expensive geometry. Second, nothing is *believed* until canonicalized geometric equality (§5.3) upgrades the necessary condition to the sufficient one. The string layer of §4 is the fast, lossy proposer; the geometry layer of §5 is the slow, exact certifier. Keeping those roles separate — and never letting a symbolic match stand in for a geometric proof — is the design's central discipline.

---

4. Strings as the Detection Engine

§3 said repetition lives on lines and comes in two flavors (periodic run, repeated substring). This section makes that computational: turn each line into a string, and the two flavors become two

classic, well-understood string problems with fast algorithms. This is the engine the whole method runs on.

4.1 Serializing a scanline into a string

Each scanline sweeps a sequence of filled and empty intervals. We turn that into a **metric-preserving token stream** — every token carries both a symbol *and* a physical length, because tolerance binning is not automatically metric-preserving and reconciliation later must operate on the physical fields [v4, §2.2]. Two encodings are produced, and both are needed:

- **Occupancy-interval encoding.** From the *merged* covered intervals along the line: filled tokens carry the quantized covered width, gap tokens the quantized empty width. Strong when features have real gaps between them.
- **Edge-boundary encoding.** From the *raw, unmerged* per-rectangle spans, each filled token keyed on `(width, edge-height)`, with **no gap token between abutted cells**. This is "every edge characterized by its projection vector," specialized to axis-aligned boxes [NOTES, decision 9].

Why two encodings — the zero-gap case. Occupancy leans on gap structure. Two directly-abutted cells with zero gap merge into one wide covered interval, so occupancy literally cannot see the repeated boundary between them; the edge encoding keeps them as two distinct tokens and recovers the array. This is not hypothetical: an earlier prototype implemented only occupancy and its standard-cell demo had to insert artificial gaps to work at all — a live demonstration that one encoding is insufficient. The C++ system implements both and recovers a genuinely zero-gap abutted array through the edge encoding (§7.1). Filled tokens get positive interned symbol ids and gap tokens negative, so downstream code reads "filled vs. empty" from the sign.

4.2 Two flavors of repetition, two families of string algorithm

With the line as a string, §3's distinction becomes a choice of algorithm — and the design documents are explicit that it is a *different tool* in each case [v4; HR, §2.2]:

- **Arrays → periodic runs.** Adjacent-on-a-lattice instances make the line a periodic run, `ABCABCABC`. The classic tool is a linear-time *runs* / maximal-periodicity algorithm (Crochemore-style): find the maximal substrings that are periodic and read off their primitive period. That period is the array's `dx` (or `dy` on a vertical line).
- **Hierarchy → repeated substrings.** Scattered instances make the line `ABC…ABC…ABC`, the same substring at unrelated offsets. That is *not* a periodic run, and calls for a different family: suffix arrays / trees for maximal repeated substrings, or grammar-induction methods (Sequitur, Re-Pair) that build a compact grammar directly from repeated structure — the latter *natively aligned* with the MDL objective, so the compression rule is not a separate pass bolted on afterward.

This is the whole array-vs-hierarchy split expressed in one sentence: **runs for the aligned case, repeated substrings for the unaligned case.** The drop-off of §3.3 lives here too — it is `R(L)/F(L)` over the *length* of a repeated substring, i.e. a property of the substring problem, not the run problem.

4.3 Why strings are the right engine

Three properties make the string reduction the right foundation rather than a convenience:

- **Sub-quadratic.** The naive alternative — compare every polygon neighborhood against every other — is quadratic in the number of primitives. String runs are linear per line; repeated-substring structures are near-linear. Candidate generation never pays the all-pairs cost.
- **Local, so partiality is free.** A run or a repeated substring is found *within* a line, over whatever sub-extent supports it. Nothing requires the line — let alone the layout — to be globally periodic. The partiality of §2.3 is the default, not a special case.
- **Compression-aligned.** Repeated-substring and grammar-induction methods are, by construction, about description-length reduction. That is the same currency as the MDL promotion rule (§5.4), so the candidate generator and the acceptance objective speak the same language.

4.4 The honest caveat: the engine is a filter, not a proof

A string match over serialized geometry is **lossy**, in two specific ways. First, **serialization order is itself a choice**: a single traversal order can hide a repeat that a different order would expose, which is why the hierarchy channel later gathers evidence *without* committing to one global order (§5.4). Second, **tolerance binning can preserve a symbolic period while the physical lattice drifts** — two tokens can bin to the same symbol while their physical lengths diverge, so a symbolically-perfect run can correspond to a geometrically-imperfect lattice. Both are reasons the string stage can only ever establish the *necessary* condition of §3. It proposes where repetition might live; §5's geometry proves whether it does. Everywhere the two could be confused, we keep the physical fields on every token and defer the verdict to geometry.

---

5. From String Candidates to Certified Structure

The engine of §4 emits candidates; this section turns them into certified arrays and hierarchies. The pipeline maps 1:1 onto the implemented C++ modules. Array detection runs §5.1–§5.3; hierarchy recovery reuses §5.1's snapping and §5.3's canonicalized-equality machinery and adds §5.4–§5.5.

5.1 Scanline construction (coordinate snapping + open-slab sampling)

Scanlines are derived from the arrangement induced by rectangle vertices, with two precision rules that matter for correctness [v4, §2.1]:

- **Coordinate snapping before slab construction.** Raw coordinates are snapped to a tolerance grid (default 0.05) and coalesced, so near-duplicate coordinates do not fragment the layout into a blizzard of near-zero-height slabs. `X`, `Y` = unique snapped x-, y-coordinates.
- **Open-slab sampling.** Consecutive `Y` values define open horizontal slabs; each non-empty slab is represented by a sample line strictly *inside* the open interval (the midpoint), never on a boundary. For rectilinear polygons the intersection topology is constant for any horizontal line strictly inside one open slab, so one representative line fully characterizes it. Horizontal edges lie on slab boundaries and are represented through the boundary coordinates directly.

Vertical slabs are built symmetrically by treating `x` as the scan-normal axis, so one code path (`build_slabs(axis_swapped)`) serves both passes. Each slab keeps two views: the **merged** covered intervals (for the occupancy encoding of §4.1) and the **raw per-rectangle spans** with their heights (for the edge encoding, which must see internal boundaries the merge would dissolve). Snapping

and open-interval sampling compose: snapping bounds the slab count, open sampling keeps each surviving slab unambiguous; neither alone suffices.

5.2 Per-line repetition and period-phase reconciliation

Per-line repetition (`analyze_row`) finds each slab's primitive physical period — the concrete realization of §4.2's run detection. The implementation uses the prototype's $O(n^2)$ self-consistency scan rather than a linear-time runs algorithm; both the design and the prototype note this stage is swappable without downstream impact [NOTES, decision 10]. The period chosen is the smallest one whose majority-vote canonical unit actually **reproduces** the row above a strict self-consistency floor — *not* merely the smallest that clears a loose match fraction. This is exactly what defeats bug 2 (§7.1): a half-period that matches only because two of four widths coincide scores below the strict floor and loses to the true primitive. The loose floor survives only to flag a genuine supercell (content near-repeats at `d` but is truly invariant only at `k·d`). Each run is recorded with both symbolic and physical fields, and its phase is φ `= x_start mod dx` using modular distance `dist_mod(a,b;p) = min(|a−b|, p−|a−b|)`.

Reconciliation (`cluster_runs`, `intersect_passes`) lifts the design's key correction into code: rows within a tile need *not* have identical content — a row crossing the top of a rectangle legitimately differs from one crossing its middle. What must agree is each row's own internal periodicity, matched on **physical step and phase**, not on symbol content [v4, §2.4]. Two runs are compatible when

$$\begin{aligned} |dx_i - dx_j| &\le \varepsilon_{\text{period}}, \\ \text{dist\_mod}(\varphi_i, \varphi_j;\, dx) &\le \varepsilon_{\text{phase}}, \\ \text{overlap}([x_{\text{start}}, x_{\text{end}}]_i,\, [\cdot]_j) &\ge w_{\min}. \end{aligned}$$

Runs are clustered by **connected components** under this hard pairwise predicate. To defeat the transitive weak-link problem the design warns about (A~B and B~C compatible does not make A~C good evidence), a robust cluster center is computed — **median** period and **circular-median** phase — and any member incompatible with that center is pruned, recomputing once [NOTES, decision 2]. The circular median has no closed form (phase wraps modulo the period); it is computed as the φ minimizing Σ `dist_mod(`φ`,` φ_{i}`; p)` searched over the observed phases plus pairwise midpoints and their antipodes [NOTES, decision 3].

The same construction runs on **vertical** slabs, producing `(dy,` ψ`)` votes. A 2D hypothesis is formed where a horizontally-supported cluster and a vertically-supported cluster overlap spatially and induce a consistent rectangle — the horizontal × vertical intersection that yields the true 2D lattice. The candidate rectangle is the **union** of member supports [NOTES, decision 4]. Clusters are classified primitive / supercell / pseudo-array; LCM combines periods only for runs already classified as genuine-supercell evidence, never as a default rule.

```
function reconcile(row_runs):
    lines = [ project(r) for r in row_runs ]         # onto (scan, normal) axes
    graph = edges { (i,j) : compatible(lines[i], lines[j]) }
    for comp in connected_components(graph):
        p_star   = median(period over comp)
        phi_star = circular_median(phase over comp; period = p_star)
        comp = { r in comp : |period(r)-p_star| <= eps_p
                             and dist_mod(phase(r),phi_star;p_star) <= eps_phi }
```

```
    if |comp| >= min_members:
        emit cluster(period=p_star, phase=phi_star,
                     support=union(scan/normal extents))
# symmetric vertical pass, then:
return intersect(horizontal_clusters, vertical_clusters)
```

5.3 Geometric verification and the defect model

Reconciliation establishes only the necessary line-wise condition of §3.1; verification (`verify_candidate`) upgrades a 2D hypothesis to a certified array by canonicalized geometric comparison against the source layout [v4, §2.5] — this is the "geometry proves" half of §4.4:

1. Clip the original geometry to the candidate rectangle.
2. Take the canonical tile as the **modal** cell content across all instances (not cell (0,0) blindly — so one defective reference cell cannot define the tile everyone is measured against).
3. Translate tile copies across `(dx, dy)` to reconstruct the rectangle.
4. Canonicalize both (snap, normalize winding, merge coincident edges, sort rings) and compare. Only matching hypotheses are accepted.

Because it is already clipping every expected instance, the same pass computes the **defect map** with Boost boolean ops:

$$\begin{aligned} \text{missing} &= \text{expected} - \text{observed} \quad \text{(instance geometry incomplete — a structural defect)} \\ \text{extra} &= \text{observed} - \text{expected} \quad \text{(routing/clutter over a complete instance)} \end{aligned}$$

An instance is **clean when none of its own geometry is missing**; overlaid extra geometry is reported separately (`n_extra`) rather than making the instance a defect [NOTES, decision 6]. This is what lets the standard-cell demo certify the array *and* report its routing overlays and its one genuine missing cell distinctly. Critically — and this is the concrete fix for bug 3 — the defect scan walks **expected physical positions** `x_start + m·dx`, looking up each position's actual geometry by physical x-window, never by token index. A missing instance is reported at its true `x`, and nothing after it desyncs [v4, §6; NOTES, decision 5].

5.4 Hierarchy recovery

Hierarchy recovery (`recover_hierarchy`) reuses §5.1's snapping and §5.3's canonicalized equality, and adds candidate generation, MDL scoring, selection, and recursion [HR, §2]. Conceptually it is §4.2's repeated-substring problem lifted into 2D.

Geometric shingles as 2D repeated substrings. Independent of any global serialization (the §4.4 order-sensitivity), a small local neighborhood — a seed rectangle plus its `k` nearest neighbors, `k = 1` by default — is gathered around each rectangle through an rtree, canonicalized to a translation- and D4-invariant signature, and hashed. Neighborhoods that hash identically across locations are candidate motifs. This is the 2D analogue of "the same substring appears at unrelated offsets": a small geometric shingle recurring at scattered positions. The seed is deliberately small — a large fixed neighborhood would fold per-instance clutter into the signature and split true instances apart [NOTES, Phase-2 decision 3]. This shingle channel is the primary, structurally sound generator because its recall does not depend on a global traversal order; the multi-serialization run/substring channel of §4.2 is a cheap but structurally incomplete supplement, left unimplemented pending an ablation [HR, §2.3; NOTES, Phase-2 decision 2].

Seed-and-extend growth — the drop-off, operationalized. Each small seed is grown geometrically: for each occurrence, add a neighboring primitive only if it is present at a **consistent offset around every occurrence simultaneously.** This is precisely the §3.3 experiment — grow the motif and watch the repeat survive or collapse. Growth stops when no further addition holds across all instances (the cliff), and is rtree-local and size-capped (`max_cell_members`) so a perfectly periodic region cannot grow an unbounded body. Because clutter is not present at a consistent offset around every instance, it cannot attach — this is what keeps clutter out of cell bodies [HR, §2.4]. The `dropoff_curve` analysis (`R(L)`, `F(L)`) exposes the same signal for inspection, and `Selection::DropOff` promotes motifs at the drop-off scale as an alternative to gain; consistent with §3.3's framing as a *scale prior*, the drop-off is a clean signal for **isolated** motifs surrounded by non-repeating content — it recovers the 2-rectangle nested motif and the 4-rectangle defective motif exactly — but weakens on a **dense lattice**, where sub-units repeat too and the cliff shifts to the array extent; there, MDL gain plus exact verification remain stronger. `Selection::MDLGain` is the default.

MDL compression gain. For a candidate body `M` occurring `k` times,

$$\text{gain}(M) = (k-1) \cdot |\text{body}(M)| \; - \; k \cdot \text{cost\_instance} \; - \; \text{def\_overhead}$$

The instance term scales with `k` (each occurrence needs its own reference); token/edge counts stand in for body cost, a first-pass model the design explicitly sanctions [HR, §2.6]. Only positive-gain candidates are promotable.

Greedy set-packing selection. Overlapping and nested candidates are expected (`ABCD`, `BCD`, `ABCDABCD` can all be valid — exactly the repeated-substring ambiguity of §3.3). Selection is weighted set-packing — maximize Σ $\texttt{gain(C}_\texttt{i}\texttt{)}$ subject to selected covers being disjoint or properly nested. This is *not* weighted interval scheduling (covers are arbitrary 2D primitive sets with no total order). We use the greedy "sort by gain, take the best, remove conflicts, repeat" heuristic — the design's sanctioned first pass — with ILP/global optimization left as future work [HR, §2.7; NOTES, Phase-2 decision 5].

Recursive nesting. After replacing selected instances with cell references, the compressed representation (references + residual) is re-tokenized and passed back through the same steps, discovering cells built from cells — a datapath built from bit-slices built from gates. Non-recursive passes under-recover structure that only becomes visible after lower-level repetition is factored out [HR, §2.8]. The recursion, greedy tie-breaks, and knob settings jointly determine *which* member of the valid-hierarchy family is returned; this is §2.4 non-uniqueness compounding across levels, expected, not a bug.

Exact rectangle-multiset certification — the correctness gate. This is the central systems contribution, not an implementation detail: it is the check that makes every recovered hierarchy trustworthy, and it is what caught the over-covering bug below. Every instance's membership is an exact index match on `(cell, orient, snapped-anchor, shape)`, and the whole result is certified by an **exact per-rectangle multiset** check: every real rectangle of `G` must be reproduced by `flatten(H)` with exactly its multiplicity. This is strictly stronger than a $\texttt{G} \subseteq \texttt{flatten(H)}$ containment test, and the difference is load-bearing. Under containment, *over-covering* passes silently — nested instances can overlap and reproduce a rectangle twice — so containment will rubber-stamp an invalid tiling; the multiset check rejects it (under-covering shows up as missing geometry, over-covering as an invalid double-count). Two mechanisms keep recovery on the valid

side of this gate: growth rejects any member that geometrically overlaps the body and maintains a per-instance *claimed-primitive* set so each primitive is claimed once, and recovery **validates after every round and reverts any round that breaks exactness** (recurse-only-while-valid). For axis-aligned rectangles under D4 this orientation-aware exact match *is* the canonicalized-equality check, so a separate per-candidate Boost equality pass is redundant. The one case not yet covered is **fractured polygons** (one logical shape as several rectangles that match only after edge-merging); reusing the array verifier's canonicalization before comparison would be needed there, and is flagged [NOTES, Phase-2 decision 6].

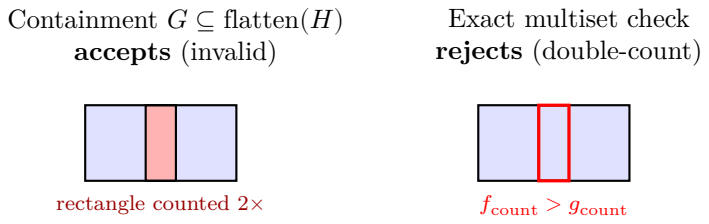


Figure 3: Why exact rectangle-multiset certification matters. Greedy growth over a dense periodic region can place two cell instances that overlap, reproducing a rectangle twice. A containment check ($G \subseteq \text{flatten}(H)$) accepts this invalid tiling; the exact multiset check requires every real rectangle to appear with exactly its multiplicity and rejects the double-count. This is the check that caught the over-covering bug.

Defect-tolerant recovery. After clean cells are established, a rescan accepts **partial** instances (a strong majority of members present, `defect_min_support`, capped by `defect_max_missing`) and records absent leaf members as a defect layer. Under the multiset gate the constraint is that every *real* rectangle of `G` is still reproduced exactly; the idealized-but-absent members are the defect layer (`flatten(H) − G`, computed by Boost), so clutter overlapping a missing member is handled. Missing *sub-cell references* are never tolerated, so nesting stays exact and clean tests are untouched.

5.5 D4 reflection and rotation

For rectilinear geometry the only rigid transforms keeping rectangles axis-aligned are the 8 elements of D4 — exactly the GDS orientation set. The `d4.hpp` module encodes these as signed permutation matrices with group composition. Grouping is by a **D4-canonical signature** (the minimum over all 8 orientations of a neighborhood's signature); matching tries all 8; growth and flatten **compose orientations through nesting** (a sub-cell reference's own orientation composes with its parent's). Every instance carries an `orient` field. This turns eight mirrored/rotated copies of a motif (Figure 4) into **one** cell definition with eight oriented instances, rather than eight unrelated cells [HR, §1.2/§5.2; NOTES, Phase-2 decision 1].

5.6 The unification: array detection = one-level recovery + lattice fit

The two problems are one. A cell whose recovered instances happen to lie on a uniform 2D lattice *is* an array — the aligned special case of §3.2. Concretely, run hierarchy recovery constrained to **one level**, take the recovered cell's instance anchors, and fit a regular axis-aligned lattice (`fit_lattice`) to them. The resulting `(dx, dy, ncols, nrows, occupied, missing)` is precisely what the dedicated array detector reports directly. §7.2 shows this holds exactly on real demo data.

Two honest wrinkles surface here, both informative. First, **unconstrained** recovery need not

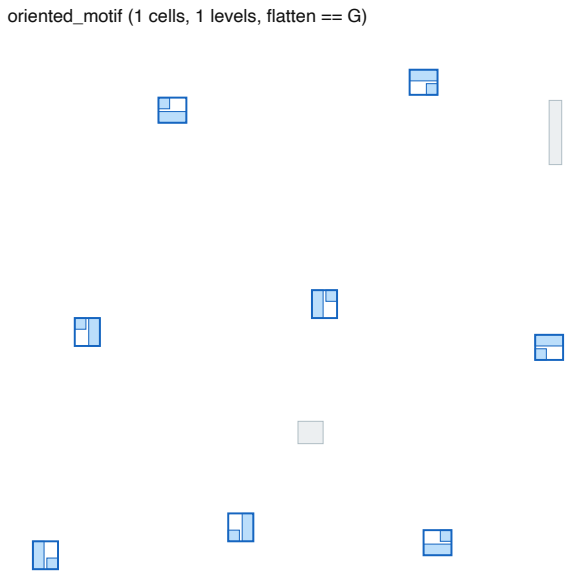


Figure 4: D4 orientation, recovered as one cell. The same L-shaped motif appears at several of the 8 axis-preserving orientations (rotations by 0/90/180/270°, each optionally mirrored), scattered off any lattice. Under translation-only matching these would be eight unrelated shapes; grouping by a D4-canonical signature collapses them to a single cell definition with oriented instances. Mirrored, power-rail-sharing standard-cell rows are the reason this matters.

return the array view: it can pick a different valid decomposition (on the standard-cell layout it returns two cells at one level rather than the single four-rectangle array tile — §7.2), so "which hierarchy" is a knob (`max_levels`, tie-breaks), not a fact — §2.4 non-uniqueness, live. Second, the correctness gate (§5.4) shapes what unconstrained recovery *can* do here: greedy MDL is tempted to keep pairing instances into a deeper binary-doubling tree over a dense array, but any round that would make instances overlap breaks the exact-multiset check and is reverted, so on dense arrays unconstrained recovery correctly stops shallow. Achieving *valid* deep compression of a dense lattice therefore needs an explicit **array node**, not greedy doubling.

The implemented array-node pass is deliberately a **compressed view over already-certified placements**, not a replacement for verification. The ordinary `top` placement list remains the authoritative flattening path; after recovery, groups of top-level placements with the same cell and D4 orientation are fit to conservative lattice nodes. A whole 2D lattice is taken when possible; otherwise the pass splits regular 1D rows/columns at non-lattice gaps, which captures datapath rows separated by block gutters. The cost model charges one array reference plus explicit missing-site exceptions, while preserving the original exact `flatten(H)` check.

---

6. Implementation

The system is C++17 on **Boost.Geometry** (header-only, permissive license, chosen over CGAL because the scope is rectilinear and CGAL's exact-kernel `Arrangement_2` machinery is more than this problem needs). Boost provides the box/polygon primitives, the boolean set operations for

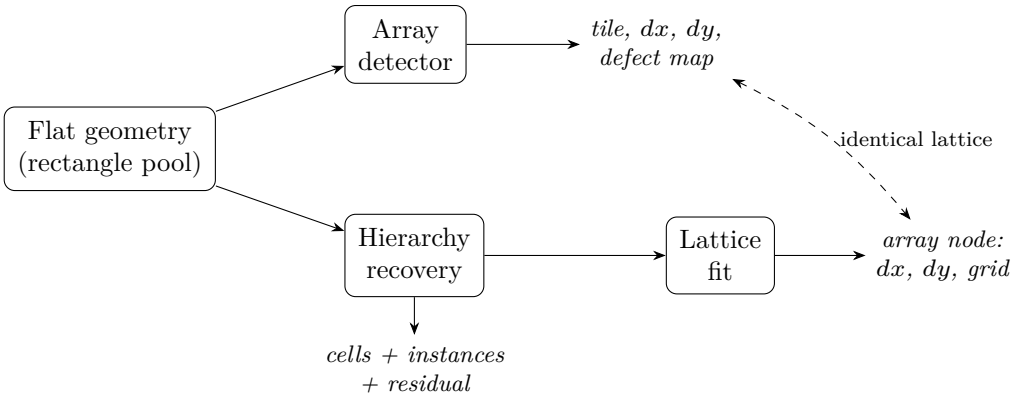


Figure 5: The unification. Both algorithms consume the same flat geometry. The array detector yields a tile, step vectors, and a defect map directly; hierarchy recovery yields cells, instances, and residual, and a *one-level* run followed by a lattice fit on the recovered instances reproduces the same lattice parameters (dashed). Array detection is the one-level, lattice-aligned special case (§5.6, §7.2).

the defect computation and canonicalized equality, and `boost::geometry::index::rtree` for the local-neighborhood queries hierarchy recovery needs. The visualizer emits SVG directly with no extra dependency.

Module layout (each maps to a design section):

File	Stage
`slab.{hpp,cpp}`	§5.1 scanlines: snapping + open-slab sampling
`encoding.{hpp,cpp}`	§4.1 both encodings (occupancy merged; edge raw/height-keyed)
`runs.{hpp,cpp}`	§5.2 per-line repetition + §5.3 defect-tolerant scan
`reconcile.{hpp,cpp}`	§5.2 compatibility predicate, CC clustering, median/circular-median, H×V intersection
`verify.{hpp,cpp}`	§5.3 canonicalized verification + missing/extra defect map
`detector.{hpp,cpp}`	array driver + dedup + candidate-reduction funnel counts
`hierarchy.hpp`	`H` = cell defs + placements + residual + defect layer
`recover.{hpp,cpp}`	§5.4 shingles → grow → MDL → greedy → recurse; `flatten(H)=G`; array-node compressed view
`d4.hpp`	§5.5 the 8 axis-preserving orientations
`bench.{hpp,cpp}`	§7.3 hierarchy-erasure benchmark generator + scorer
`svg / hr_svg`	visualization

Performance-relevant choices (§7.4 quantifies them): rtree neighbor queries for shingle k-NN and growth candidates (replacing an $O(n^2)$ per-seed distance scan); an **integer-keyed spatial hash** (FNV over `(cell, orient, snapped-anchor)`) instead of `std::string` keys in the hot lookup path; rtree-local, size-capped growth; and an **O(n) exact rectangle-multiset** verification that groups and reconstructs existing rectangles exactly (never merging or refracturing) and compares multiplicities by hashing snapped rectangles, instead of the iterated Boost `union_` that made the first correctness-focused pass $O(n^2)$.

Verification is a correctness gate, not just a metric — and a linear one. The O(n) multiset check is what makes recovery trustworthy at scale, and it is also what *caught* the over-covering bugs. An earlier version relaxed the check to `G` ⊆ `flatten(H)` (containment), which is cheap but too weak: it accepts a hierarchy in which two nested instances overlap and reproduce a rectangle twice. That masked a real growth bug — in a perfectly periodic region, seed-and-extend would add a neighbor's rectangle "present at all instances," making adjacent cell instances overlap — and inflated the reported compression, because the invalid, over-covering hierarchy looked smaller than any valid one. Switching to the exact multiset check surfaced the bug, and combined with the growth fix (overlap rejection + per-instance claimed-primitive sets) and recurse-only-while-valid, it both restored correctness and *lowered* the honest compression numbers (§7.3, §7.5). The check stays O(n): grouping and multiset comparison are hashing passes over snapped rectangles, no Boost boolean union in the hot path.

---

7. Evaluation

All numbers below are from actual runs of the implemented system, including the §7.5 real-GDS round-trip; the only open item is the real SKY130/ORFS crop, marked `[TBD]`.

7.1 Correctness: bug-class regression tests and demos

The three bug classes diagnosed in the Python prototype are reproduced as passing regression tests (`test_bugs`), plus the closed encoding gap:

- **Bug 1 (symbol collision).** A layout where non-repeating clutter has, by coincidence, the exact width of a real repeated token. The detector still finds `dx = 20` (the true period), the 8×6 grid, and excludes the colliding clutter; all 48 instances are clean. Non-periodic content is trimmed against the canonical unit and geometric verification is mandatory, so a single-token width coincidence can neither form a period nor survive verification.
- **Bug 2 (spurious sub-period from equal widths).** A repeating unit with an internal repeated sub-value. The detector finds `dx` ≈ `78` (the full primitive period), *not* the ≈ `39` half-period, because the period is chosen by self-consistency relative to the best achievable, not an absolute floor (§5.2).
- **Bug 3 (phase attribution across a defect).** A grid with a genuine missing instance in the middle. Of 48 expected instances, exactly 47 are clean and exactly one defect (type MISSING) is reported, **attributed to column 3, row 2 — not shifted right**. The row-scan test confirms the mechanism: period `dx = 20`, 8 expected positions, one defect at physical `x = 60`, not shifted. The defect scan walks expected physical positions, never token indices (§5.3).
- **Zero-gap gap closure.** A directly-abutted standard-cell array (which occupancy alone cannot see) is detected via the edge encoding (§4.1).

Array-detection demos (`demo`), with the candidate-reduction funnel `N` symbolic runs ≫ `M` clusters ≫ `K` verified arrays:

Demo	Rects	Runs N	Clusters M	Verified K	dx	dy	Grid	Clean / Missing / Extra
bauhaus (decorative)	101	58	2	1	40	40	8×6	47 / 1 / 0
standard-cell (gapped rows)	109	109	5	1	87	26	5×5	24 / 1 / 15
standard-cell (zero-gap abutted)	102	45	2	1	79	26	5×5	24 / 1 / 0

The gapped standard-cell demo (Figure 2) carries 15 routing overlays (`extra`) and one genuine missing cell (`missing`, area 356.4), reported distinctly — the value of the defect model. The abutted demo isolates the encoding point: edge encoding recovers zero-gap geometry occupancy cannot. (The abutted demo deliberately uses non-crossing border clutter, because edge-only encoding is clutter-fragile over zero-gap geometry — see §8.)

Hierarchy-recovery demos (`hr_demo`):

Demo	Leaf rects	Cells	Levels	Result
nested motif (irregular placement)	29	2	2	leaf cell ×13, super-cell (3 refs to leaf) ×3, 7 top, 3 residual; 1.93×
oriented motif (all 8 D4)	18	1	1	one cell, 8 instances spanning o0..o7; 1.50×
defective motif	20	1	1	one 4-member cell ×5, one instance missing a member; defect area 120

All three satisfy `G ⊆ flatten(H)`. The nested and oriented cases are Figures 1(b) and 4: unaligned, scattered repetition recovered without any lattice to lean on. The oriented case is the D4 payoff; the defective case exercises defect-tolerant recovery end to end.

7.2 The unification result

On identical flat input, the dedicated array detector and one-level hierarchy recovery + lattice fit (`unify_demo`, `test_unify`) produce identical lattices:

Layout	Path	dx	dy	Grid	Occupied	Missing	flatten=G
bauhaus (101 rects)	array detector	40	40	8×6	47	1	—
bauhaus	1-level recovery + lattice fit	40	40	8×6	47	1	yes

Layout	Path	dx	dy	Grid	Occupied	Missing	flatten=G
standard-cell (109 rects)	array detector	87	26	5×5	24	1	—
standard-cell	1-level recovery + lattice fit	87	26	5×5	23	2	yes

`test_unify` asserts these match: array `dx`/`dy`/grid equal the lattice-fit values, and occupied + missing equals the array instance count. (On standard-cell the two paths partition the same sites slightly differently between "occupied" and "missing" — 23+2 vs 24+1 — because one-level recovery groups by exact modal-cell membership while the array verifier's modal-tile clip counts one borderline instance as clean; both agree on the lattice and both satisfy their correctness constraint.) The unconstrained full-hierarchy path finds a *different* valid decomposition on each layout — bauhaus: 1 cell at 1 level, cost 56 vs. flat 101 (1.80×); standard-cell: 2 cells at 1 level, cost 43 vs. 109 (2.53×) — a live instance of §2.4 non-uniqueness. **This equality between the two independent paths is the paper's core empirical claim: array detection is the one-level lattice-aligned special case of hierarchy recovery, demonstrated, not asserted.**

7.3 Hierarchy-erasure benchmark

The benchmark (`bench`, `erasure_demo`) builds a design with **known** hierarchy — a leaf 4-rectangle "gate" → a horizontal array "slice" → a vertical stack of alternately-mirrored slices "block" → a row of blocks "chip" — flattens it to raw rectangles, strips all cell/instance/array metadata, and asks the recoverer to reconstruct a compact hierarchy from geometry alone. Ground truth is the design's own construction, so nothing is hand-labeled. We report two costs: the placement-list hierarchy cost (`flat leaf count / hier_cost`) and the array-node view (`flat leaf count / array_cost`), where regular placement grids cost one array reference plus explicit missing-site exceptions.

Case	Rects	Cells	Levels	Top insts	Array nodes	Residual	Defective	Placement-list comp.	Array-node comp.	flatten=G (exact)	Leaf found
flat array (1 block)	96	1	1	24	1	0	0	**3.43**×	**19.20**×	YES	found
nested ×2 blocks	196	1	1	47	8	8	0	**3.32**×	**9.33**×	YES	found
nested + mirrored rows	246	1	1	59	10	10	0	**3.37**×	**9.84**×	YES	found
larger + mirrored + clutter	588	1	1	142	18	20	0	**3.54**×	**13.36**×	YES	found

Every case satisfies the exact rectangle-multiset flatten check (not merely containment) and recovers the ground-truth leaf gate. As a plain placement-list hierarchy the valid compression is ~**3.3**–**3.5**×.

Adding the array-node view raises this to ~**9.3**–**19.2**× by charging regular placement rows/grids as explicit array references instead of as individual top placements. The mirrored cases exercise D4 (adjacent rows share a rail). These numbers replace an earlier, higher placement-list figure (~5–8×) that came from the over-covering verification bug described in §6: under the weak `G` ⊆ `flatten` check, invalid hierarchies whose nested instances overlapped were accepted and looked artificially smaller. The corrected exact-multiset gate rejects those; array nodes then recover the dense-lattice compression through a valid representation rather than through overlapping hierarchy. Defect tolerance is also demonstrated: on a defective datapath the missing gate member is recorded as the defect layer while every real rectangle is still reproduced exactly.

7.4 Performance and scalability

The recovery pipeline was profiled after a first correctness-focused pass revealed an $O(n^2)$ blowup (a ~1000-rectangle design took minutes). Phase-by-phase timing showed **the algorithm was never the bottleneck**: the per-round work (shingle / group / build / dedup / greedy) was already linear and fast — about **6 ms at n = 3072**. The entire $O(n^2)$ lived in the final `flatten(H) = G` verification, which built one giant multipolygon by iterated Boost `union_`.

The four fixes (rtree neighbor queries, integer-keyed spatial hash, rtree-local size-capped growth, and the O(n) exact rectangle-multiset verification — §6) removed the quadratic term entirely:

Metric	Value
Per-rectangle cost	~3 μs/rect, linear
Per-round work at n = 3072	~6 ms
1.58 M rectangles, single round	~4.8 s (one core)
1.58 M rectangles, full 8-level recursion	~30 s (one core)
Correctness (exact rectangle-multiset flatten check)	intact at all sizes

The key methodological finding: **the interesting bottleneck was verification, not detection.** Once verification is made linear, the whole pipeline is linear — and, as §6 recounts, making that verification *exact* (rather than a cheaper containment test) is also what caught the over-covering bug, so the correctness gate and the performance fix are the same piece of code.

Path to billions (architecture ready, not yet built): **tiling + parallelism** (rounds, tiles, and candidates are independent; the rtree and per-tile recovery parallelize cleanly, with a stitch pass for motifs crossing tile borders); **streaming / out-of-core** for layouts exceeding RAM; richer **array-node packing** (nested array nodes, sparse occupancy bitmaps, and stitchable row/column arrays across tile boundaries); and applying the same O(n) rectangle approach to the array detector's own verifier, which still uses the iterated-union pattern.

7.5 Real-GDS hierarchy-erasure

The real-GDS round-trip exercises genuine GDS I/O and real flattening. `scripts/gds_roundtrip.py` uses `gdstk` to **write** a genuinely hierarchical `.gds`: a leaf `gate` cell (4 rectangles), a `slice` built as an array reference (AREF) of the gate, a `block` of alternately `x_reflection`-mirrored slices (the way place-and-route mirrors standard-cell rows to share a power rail), a `top` of blocks, plus routing clutter. It then **reads the file back and flattens it with the tool**, discarding all cell/instance/array metadata, and exports the flat rectangles. `build/gds_recover` recovers a hierarchy from those flat rectangles alone. The geometry is synthetic, but the GDS format, the

hierarchy, the mirroring, and the flatten are all real — and dropping in a real **SKY130**/ORFS `.gds` uses the identical read → flatten → export path.

Quantity	Value
Flattened rectangles (G)	772
Ground truth	4 designer cells (gate/slice/block/top), 4 levels, gate = 4 rects × **192** instances
Recovered	**1 cell of 4 rectangles**, 1 level, **191** top placements, 8 residual rectangles
Leaf gate recovered	YES (4-rectangle cell)
flatten(H) = G	**YES, exact** (rectangle-multiset check)
Placement-list compression	**3.80**× (flat 772 → hierarchy 203)
Array-node compression	**20.86**× (flat 772 → array-hierarchy 37, 24 array nodes)

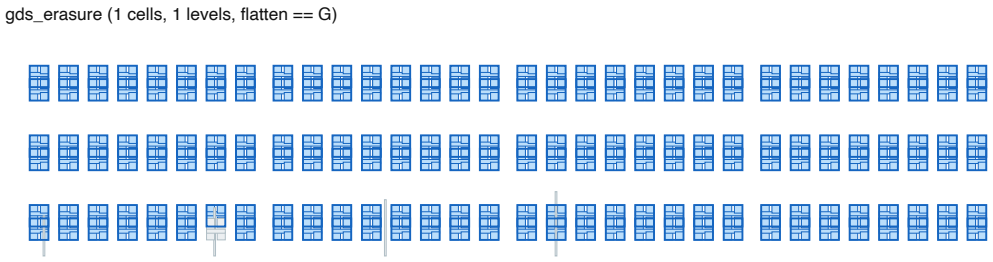


Figure 6: Recovery from a flattened, metadata-stripped real GDS. The hierarchical `.gds` (gate → slice → block → top, with mirrored rows) is written and flattened by `gdstk`; the recoverer reconstructs the 4-rectangle gate cell as instances across the layout (blue), with routing clutter left as residual (gray), `flatten(H) = G` exact.

The recoverer reconstructs the ground-truth 4-rectangle gate cell at **191 of 192** instances; one edge gate falls to residual, and the clutter is left as residual — exactly the kind of principled disagreement §2.4 predicts. We therefore report this as **agreement, not accuracy** [HR, §1.3]: the recovered hierarchy is a compact, geometrically exact decomposition (`flatten(H) = G` holds exactly under the multiset gate), which need not match the designer's four-level decomposition. The placement-list hierarchy gives 3.80× compression, at the top of the corrected ~3.3–3.8× placement-list range; the array-node view identifies 24 regular row arrays and raises the valid compressed view to **20.86**×.

This is also the evaluation that surfaced the over-covering verification bug (§6): the dense, mirrored, abutting arrays in a real flattened GDS are precisely the stress case that made adjacent-instance overlap possible, which the exact-multiset gate then rejected.

The remaining source per the design's three-source scoping recommendation [v4, §7.3] — a real flattened crop from the **SkyWater SKY130** PDK / **OpenROAD-flow-scripts (ORFS)** flow — plugs into the same path; `FreePDK45`/Nangate-style libraries are a classic academic baseline and `ALIGN` analog layouts a harder stretch target. §7.6 takes the first step on real SKY130 silicon (a direct crop rather than the erasure harness, since a fabricated macro carries no synthetic per-layer ground truth).

7.6 Real-SKY130 crop: recovery and its honest limits

The §7.5 round-trip's GDS I/O is real but its geometry is synthetic. To close the last gap we ran the recoverer on genuinely industrial layout: a **SkyWater SKY130** 1 KB dual-port SRAM macro generated by **OpenRAM** — a 9.5 MB hierarchical GDS whose bitcell array places **8192** instances of one `openram_dp_cell` in four D4 mirror orientations on a 6.24 μm × 3.95 μm lattice. `scripts/crop_sram_layer.py` crops one array cell, one mask layer, and a spatial window to the same flat-rectangle format and strips all metadata, so the recoverer sees only rectangles. This also exercises the first implementation of the §9 "next step" — a separate `recover_nested` pass that promotes recovered placements into named nested cells (`slice`/`block`) whose bodies hold array references, certified by the same exact-multiset gate.

On a **single bitcell row** the recoverer works cleanly and exactly. For the met1 layer of a 64-cell row (2048 rectangles) it recovers the repeated per-bitcell met1 tile as a leaf cell and arrays it into two mirror-orientation `slice` cells, `flatten(H) = G` exact:

Layer (single row)	Rects	Placement-list	Nested
met1 (64 cells)	2048	**21.1**×	**55.4**×
met1 (15 cells)	928	15.0×	25.1×
li1 (15 cells)	696	9.5×	11.4×
poly (15 cells)	116	3.5×	14.5×

The recovered cell is a valid *single-layer* projection of the designer's bitcell, not the full multi-layer cell — **agreement, not accuracy** (§1.3) on real geometry.

Two limits surfaced, both structural rather than incidental. First, **full 2D recovery of the dense array reverts to nothing**: the bitcell's own per-layer extent (≈ 4.2 μm diagonal on met1) *exceeds* the 3.95 μm vertical row pitch, so no seed-and-extend radius separates a mirrored cell from its abutting vertical neighbor — every candidate over-grows, and the exact-multiset gate (§5.4) correctly reverts it, leaving zero cells. This is the §7.3/§9 dense-array hardness made concrete on silicon: when a tile is packed tighter than its own extent, geometry-first growth needs the lattice prior (a super-cell placed only at run origins), which the 1D row supplies — its 6.24 μm column pitch does exceed the tile extent — but the 2D packing does not. Second, the **uniform contact layer (licon) is not recovered at all**: its rectangles are all identical squares, so the local geometric shingle (§5.4) is ambiguous — 1131 contacts collapse to five signature groups and no stable tile forms. This is the §4.4 lossiness from the other side: the deliberately-asymmetric synthetic gate is the easy case; a real field of identical contacts is the adversarial one. Metal and poly layers, with their varied per-cell shapes, recover; the contact layer does not. The honest reading: on real SKY130 geometry the framework recovers and compactly nests a repeated cell **line-wise and exactly** (up to 55× on met1), while 2D dense-array recovery awaits the array-node lattice prior driving growth, not just scoring it.

---

8. Related Work

We situate the framework against three families, briefly and honestly — each is a lens on the string-vs-geometry division of labor the paper is built on.

Global-periodicity methods. Autocorrelation, Fourier-peak, and lattice-from-image techniques recover a single dominant period for an image or point set. They are efficient and well-understood, but they assume global periodicity. Our target is the opposite regime — a partial array embedded in a mostly non-repeating layout (§2.3) — where a global period either does not exist or is dominated by clutter. We use periodicity only *line-wise and locally* (§3.1), as candidate evidence, never as a global assumption.

Pairwise geometric motif matching. One can search directly for repeated geometry by comparing polygon neighborhoods against each other. This is expressive but tends toward quadratic cost in the number of primitives, and needs a canonicalization to be robust. We keep the canonicalized geometric comparison (it is our certifier, §5.3) but avoid the quadratic candidate search: array candidates come from line-wise runs (linear per line), and hierarchy candidates come from local geometric *shingles* gathered through an rtree (§5.4), so candidate generation never pays the all-pairs cost — the sub-quadratic argument of §4.3.

String maximal-repetition / grammar induction. Linear-time *runs* algorithms find maximal periodic substrings; suffix structures and grammar-induction methods (Sequitur, Re-Pair) find repeated (not necessarily contiguous) substrings and are natively aligned with a compression objective. We borrow these as the *serialization-channel* candidate generators of §4.2, with the explicit tool distinction the design documents make: periodic *runs* fit adjacent-on-a-lattice array instances, whereas *repeated substrings* fit cell instances scattered with unrelated content between them. Crucially, a string match over serialized geometry is lossy (§4.4) — so the string stage is a filter and the geometry stage is the proof.

The distinguishing stance of this work is the combination: **partial** structure in clutter, **sub-quadratic** candidate generation via line-runs and local shingles, **canonicalized geometric certification**, and an **MDL** objective — with array detection falling out as a special case rather than being treated as a separate problem.

---

9. Limitations and Non-Uniqueness

We follow the design documents in being explicit about what the framework does *not* claim.

Recovered hierarchy is one of many valid decompositions. This is the central honesty point [HR, §1.3, §2.8]. A flat layout admits many geometrically valid hierarchies; both the greedy selection (order-dependent) and the recursion mean the output is one member of a family, not a canonical answer. We saw it live in §7.2 (unconstrained recovery picks a different decomposition from the array view) and §7.5 (191 of 192 gates recovered, the 192nd falling to residual). Where a source hierarchy is known we therefore report **agreement, not accuracy**. The knobs (`max_levels`, `min_cell_members`, `cost_instance`, `gain_min`, …) select *which* valid hierarchy is returned; they are non-uniqueness controls, not accuracy dials.

Greedy selection is non-optimal. Selection is weighted set-packing (NP-hard in general); the greedy heuristic has no optimality guarantee. ILP for small/medium candidate counts, and a principled global objective, are explicit future work [HR, §2.7].

Edge-only encoding is clutter-fragile. Occupancy leans on gap structure to stay robust to routing wires crossing an array (the gapped standard-cell demo has crossing wires and detects perfectly). The edge encoding has no gaps to anchor on, so clutter crossing **zero-gap** geometry

perturbs the token stream and can push individual rows to spurious periods — a concrete instance of the §4.4 lossiness. The abutted demo therefore uses non-crossing border clutter, isolating the point it exists to make. An occupancy-vs-edge ablation over routed real layouts is open [NOTES, "One honest limitation"; v4, §9 Q9].

Fractured-polygon verification not yet implemented. Recovery's exact rectangle-multiset match certifies one logical shape only when its rectangles match without edge-merging. A shape fractured differently between two instances would need the array verifier's canonicalization reused in the recovery path — flagged, not built [NOTES, Phase-2 decision 6].

Full hierarchy extraction: a first nested extractor, with real limits. Beyond the base recoverer's two compression views (placement-list and array-node), a first pass of the next step is now implemented as a separate `recover_nested` path: it promotes repeated placement runs into named intermediate `slice`/`block` cells whose bodies hold array references, recursing on the mixed representation (`leaf refs + array refs + residual`) under the same exact-multiset gate. On the synthetic datapaths it lifts the placement-list ~3.3–3.5× to a nested **9.5–14**× (`gate` → `slice` → `block` → `top`, exact), and on a real SKY130 SRAM row to **55**× (§7.6). It is still **not full source-like extraction**. Two simplifications remain: nested cells are placed at orientation 0 with mirroring baked into distinct cell bodies (so a mirrored row is a second `slice` cell rather than one cell reused under D4), and — the load-bearing gap — 2D dense arrays remain blocked by the tile-extent-exceeds-pitch limit of §7.6, because growth is still seed-and-extend and only the *scoring*, not the *candidate generation*, uses the lattice. Closing it means driving growth from the array-node prior (place a super-cell at run origins), so a tile packed tighter than its own extent is still separable.

Other scoped-out items. Multi-layer / multi-class geometry, holes and overlapping polygons, oblique (non-orthogonal) lattices, and the §4 candidate-scoring formula (normalized-product vs. log-weighted-sum, left unimplemented because verification alone suffices at demo scale [NOTES, decision 8]) are all out of the current scope.

---

10. Conclusion and Future Work

The paper is built on one observation and the algorithm it forces. The observation: repetition in a 2D layout shows up as repetition on 1D lines — periodic runs where a cell is lattice-aligned, repeated substrings where it is scattered — so line-wise 1D repetition is a *necessary condition* for the structure we want to recover, cheap to test and cheap to falsify. The algorithm: serialize each scanline into a string and let fast string-repetition algorithms be the detection engine, with the repeat-length drop-off fixing the cell *scale* where a lattice cannot. Because the string layer is lossy, it only ever *proposes*; canonicalized geometric equality *proves*, and an MDL objective decides which proven repeats become cells. The unification is not rhetorical — array detection is realized as one-level hierarchy recovery plus a lattice fit, and the two independent paths produce identical lattice parameters on real demo data (§7.2).

The implemented system reproduces three previously-diagnosed bug classes as passing regression tests, handles zero-gap abutted geometry through a second encoding, recovers D4-oriented instances as one cell, and passes both a synthetic and a real-GDS hierarchy-erasure benchmark with exact rectangle-multiset flatten-equality. As a placement-list hierarchy it achieves ~3.3–3.8× compression while recovering the leaf cell in every benchmark; with the new array-node compressed view it reaches ~9.3–20.86× on dense datapaths by representing regular top-level placement groups ex-

plicitly. It runs linearly at ~3 μs/rect (1.58 M rectangles in ~4.8 s single-round on one core). A methodological point worth carrying forward: the exact verification is a correctness gate, not a scoreboard — it caught an over-covering bug that a cheaper containment check had hidden, and lowering the placement-list compression to the honest range was the *right* outcome. The framework is honest about non-uniqueness throughout: agreement, not accuracy.

Future work, in rough priority order: (1) extend the real-SKY130 evaluation (§7.6) from a 1D bitcell row to the full 2D dense array and to an ORFS erasure crop — the 2D case is blocked until the array-node lattice prior *drives* growth (super-cell placed at run origins) rather than only scoring it; (2) **complete the nested extractor** — the §7.6 prototype recovers `slice`/`block` cells and nests exactly on synthetic and real rows, but still needs D4-aware single-cell reuse for mirrored rows (rather than a second cell) and multi-layer tiles; (3) **tiling + parallelism** and streaming toward billion-polygon layouts; (4) **fractured-polygon verification** in the recovery path via reused canonicalization; (5) the **multi-serialization** run/substring generator of §4.2 and an occupancy-vs-edge ablation; (6) ILP/global selection to replace greedy; and (7) multi-layer support. Papers on either algorithm should follow real, benchmarked output — this draft is the working record toward that, not a finished result.

References

- [**v4**] *Geometry-First Array Detection in 2D Polygon Layouts via 1D Repetition Hypotheses*, v4 (`repeat-cell-detection-proposal-v4.md`). The array-detection design of record.
- [**HR**] *Hierarchy Recovery from Flattened Polygon Layouts via Repetition-Driven Compression*, v1 (`hierarchy-recovery-proposal-v1.md`). The companion hierarchy-recovery design.
- [**NOTES**] *Phase 1 — implementation notes & flagged design decisions* (`NOTES.md`). Design decisions taken where the proposals left choices open, measured performance, and honest limitations.
- SkyWater **SKY130** PDK: https://skywater-pdk.readthedocs.io/en/main/contents.html
- **OpenROAD-flow-scripts (ORFS)**: https://openroad-flow-scripts.readthedocs.io/en/latest/
- OpenROAD Project (background): https://en.wikipedia.org/wiki/OpenROAD_Project
- **ALIGN** (analog layout automation): https://align-analoglayout.github.io/ALIGN-public/
- **FreePDK45 / Nangate**-style open standard-cell libraries (classic academic baseline).
- **DeepPCB** (secondary raster baseline only): https://github.com/tangsanli5201/DeepPCB